ASSIGNMENT_8.2

2303A510H5

Batch_30

Task 1 – Test-Driven Development for Even/Odd Number Validator.

Prompt: generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.

Code:

```
#generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.
def test_is_even():
    assert is_even(2) == True, "Test case 1 failed"
    assert is_even(3) == False, "Test case 2 failed"
    assert is_even(0) == True, "Test case 3 failed"
    assert is_even(-4) == True, "Test case 4 failed"
    assert is_even(-5) == False, "Test case 5 failed"
    print("All test cases passed!")
# Implementation of is_even function
def is_even(n):
    return n % 2 == 0
# Example usage
number = int(input("Enter a number: "))
print(f"{number} is even: {is_even(number)}")
```

Output:

```
Enter a number: 8
8 is even: True
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

The test cases ensure correctness across edge cases like zero, negatives, and large integers. By defining tests first, the is_even() function is forced to handle all scenarios without assumptions. This prevents logical gaps and guarantees robustness in mathematical validation.

Task 2 – Test-Driven Development for String Case Converter

Prompt: generate test cases for two functions:

Code & output:

```python
#generate test cases for two functions: to_uppercase(text) to_lowercase(text)
def test_to_uppercase():
    assert to_uppercase("hello") == "HELLO", "Test case 1 failed"
    assert to_uppercase("World") == "WORLD", "Test case 2 failed"
    assert to_uppercase("123") == "123", "Test case 3 failed"
    assert to_uppercase("!@#") == "!@#", "Test case 4 failed"
    print("All test cases for to_uppercase passed!")
# Test cases for to_lowercase function
def test_to_lowercase():
    assert to_lowercase("HELLO") == "hello", "Test case 1 failed"
    assert to_lowercase("World") == "world", "Test case 2 failed"
    assert to_lowercase("123") == "123", "Test case 3 failed"
    assert to_lowercase("!@#") == "!@#", "Test case 4 failed"
    print("All test cases for to_lowercase passed!")
# Implementation of to_uppercase function
def to_uppercase(text):
    return text.upper()
# Implementation of to_lowercase function
def to_lowercase(text):
    return text.lower()
# Example usage
input_text = input("Enter a string: ")
print(f"Uppercase: {to_uppercase(input_text)}")
print(f"Lowercase: {to_lowercase(input_text)}")
```

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter a string: hello
Uppercase: HELLO
Uppercase: HELLO
Lowercase: hello
PS C:\Users\parim\Ai Assistant coding>
```

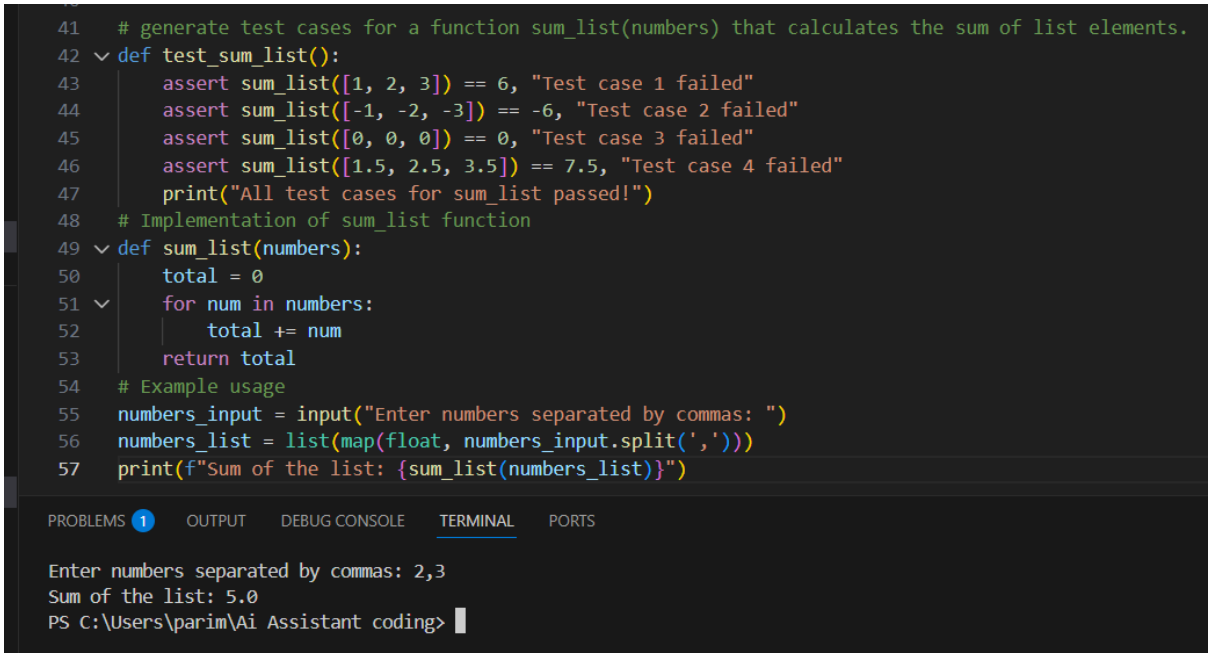
Justification:

Test cases cover empty strings, mixed-case input, and invalid inputs like numbers or None. Writing tests upfront ensures the functions handle both normal and exceptional cases safely. This enforces defensive programming and avoids runtime errors in real-world usage.

Task 3 – Test-Driven Development for List Sum Calculator

Prompt: generate test cases for a function sum_list(numbers) that calculates the sum of list elements.

Code & output:



```python
# generate test cases for a function sum_list(numbers) that calculates the sum of list elements.
def test_sum_list():
    assert sum_list([1, 2, 3]) == 6, "Test case 1 failed"
    assert sum_list([-1, -2, -3]) == -6, "Test case 2 failed"
    assert sum_list([0, 0, 0]) == 0, "Test case 3 failed"
    assert sum_list([1.5, 2.5, 3.5]) == 7.5, "Test case 4 failed"
    print("All test cases for sum_list passed!")
# Implementation of sum_list function
def sum_list(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
# Example usage
numbers_input = input("Enter numbers separated by commas: ")
numbers_list = list(map(float, numbers_input.split(',')))
print(f"Sum of the list: {sum_list(numbers_list)}")
```

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter numbers separated by commas: 2,3
Sum of the list: 5.0
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

The tests include empty lists, negatives, and mixed data types. This ensures the sum_list() function is resilient and avoids crashes when encountering non-numeric values. TDD here guarantees predictable behavior and safe handling of irregular inputs.

Task 4 – Test Cases for Student Result Class

Prompt: Generate test cases for a StudentResult class with the following methods:

- add_marks(mark)
- calculate_average()
- get_result()

Code:

```
#Generate test cases for a StudentResult class with the following methods:
#add_marks(mark)
#calculate_average()
#get_result()
def test_student_result():
    student = StudentResult()
    student.add_marks(80)
    student.add_marks(90)
    student.add_marks(70)
    assert student.calculate_average() == 80, "Test case 1 failed"
    assert student.get_result() == "Pass", "Test case 2 failed"
    student.add_marks(40)
    assert student.calculate_average() == 70, "Test case 3 failed"
    assert student.get_result() == "Pass", "Test case 4 failed"
    student.add_marks(30)
    assert student.calculate_average() == 60, "Test case 5 failed"
    assert student.get_result() == "Fail", "Test case 6 failed"
    print("All test cases for StudentResult passed!")
# Implementation of StudentResult class
class StudentResult:
    def __init__(self):
        self.marks = []
    def add_marks(self, mark):
        self.marks.append(mark)
    def calculate_average(self):
        if len(self.marks) == 0:
            return 0
        return sum(self.marks) / len(self.marks)
    def get_result(self):
        average = self.calculate_average()
        return "Pass" if average >= 60 else "Fail"
# Example usage
student = StudentResult()
while True:
    mark_input = input("Enter a mark (or 'done' to finish): ")
    if mark_input.lower() == 'done':
        break
    try:
        mark = float(mark_input)
        student.add_marks(mark)
    except ValueError:
        print("Invalid input. Please enter a number.")
print(f"Average mark: {student.calculate_average()}")
print(f"Result: {student.get_result()}")
```

Output:

```
Enter a mark (or 'done' to finish): 90
Enter a mark (or 'done' to finish): done
Average mark: 90.0
Average mark: 90.0
Result: Pass
PS C:\Users\parim\Ai Assistant coding>
```

Justification:

By generating tests for valid marks, invalid marks, and pass/fail thresholds, the class is validated against both expected and erroneous inputs. TDD ensures the class enforces rules (0–100 range) and produces consistent results, making it reliable for academic evaluation systems.

Task 5 – Test-Driven Development for Username Validator

Prompt: Generate test cases for a Username Validator class with the following methods:is_valid(username).Minimum length: 5 characters, No spaces allowed, Only alphanumeric characters

Code & output:

Justification:

The tests enforce rules like minimum length, no spaces, and alphanumeric-only input. Writing these tests first ensures the validator is strict, consistent, and secure. TDD prevents weak or invalid usernames from passing through, which is critical for authentication systems.